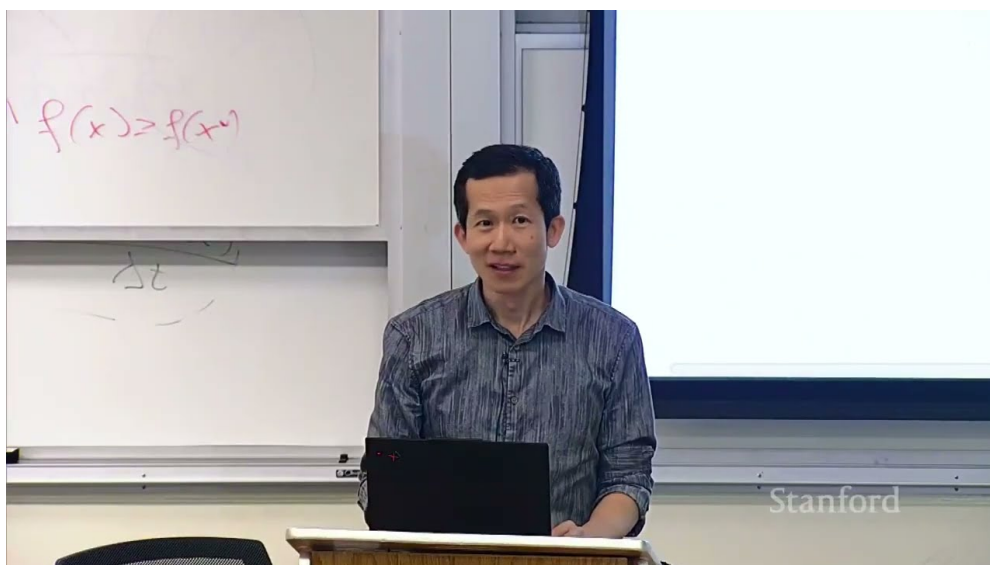


课程笔记

CS336 2026 Lecture 1: 课程导论、语言模型 版图与 Tokenization

基于 Stanford CS336 Spring 2026 官方可执行讲义整理

2026 年春季



课程/频道: Stanford CS336

发布日期: Spring 2026

材料类型: 官方可执行讲义 lecture01-slides.py

课程主页: <https://cs336.stanford.edu/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 如何阅读这一讲	2
1.1 Spring 2026 的新版定位	2
1.2 官方材料的形式：可执行讲义	2
1.3 本章小结	3
2 为什么今天还要从零理解语言模型	3
2.1 抽象层上移带来的生产力与风险	3
2.2 从 2016 到今天：研究方式的迁移	4
2.3 本章小结	4
3 工业化语言模型与课堂可达性的边界	4
3.1 闭源报告与不可复现细节	5
3.2 尺度改变系统瓶颈	5
3.3 尺度改变能力表现	6
3.4 三类可迁移知识	7
3.5 Bitter Lesson 的正确读法	8
3.6 本章小结	8
4 语言模型发展版图	9
4.1 Pre-neural：熵、n-gram 与传统系统	9
4.2 Neural ingredients：2010 年代的关键积木	9
4.3 Early foundation models 与 scaling	9
4.4 开放模型：从 open-weight 到 open development	10
4.5 语言模型形态的变化	10
4.6 本章小结	11
5 课程结构：五个单元如何构成语言模型栈	11
5.1 Basics：从 tokenizer 到训练循环	11
5.2 Systems：硬件不是背景板	12
5.3 Scaling laws：用小规模实验规划大规模训练	13
5.4 Data：数据处理是模型质量的一部分	14
5.5 Alignment：从 next-token prediction 到弱监督优化	14
5.6 本章小结	15
6 Efficiency 作为贯穿全课的设计语言	15
6.1 资源的多重含义	15
6.2 Tokenization 也是 efficiency 决策	15
6.3 本章小结	16
7 Tokenization 的基本定义	16
7.1 Tokenizer 抽象接口	16
7.2 Round-trip：可逆性是底线	17
7.3 Compression ratio：bytes per token	17

7.4	GPT-5 tokenizer 的观察	17
7.5	本章小结	18
8	三种朴素 tokenizer 及其失败方式	18
8.1	Character tokenizer: 直观但词表尴尬	18
8.2	Byte tokenizer: 覆盖性强但序列太长	18
8.3	Word tokenizer: 语义自然但开放词表难处理	19
8.4	一个具体例子: 三种方案的数值直觉	19
8.5	本章小结	19
9	Byte Pair Encoding: 从压缩算法到语言模型 tokenizer	20
9.1	BPE 的基本数据结构	20
9.2	merge 函数的细节	20
9.3	训练 BPE: 统计相邻 pair	21
9.4	Worked example: the cat in the hat	21
9.5	使用 BPE tokenizer	22
9.6	本章小结	22
10	Assignment 1 中 tokenizer 实现的工程细节	23
10.1	不要每次遍历所有 merges	23
10.2	Special tokens 不能被普通规则吞掉	23
10.3	Pre-tokenization: 先切粗块, 再做 BPE	23
10.4	Decode 的错误处理	23
10.5	本章小结	24
11	Tokenizer-free 梦想与现实约束	24
11.1	为什么直接建模 bytes 很诱人	24
11.2	BPE 为什么仍然实用	24
11.3	本章小结	25
12	拓展阅读	25
12.1	本章小结	25
13	总结与延伸	25
13.1	核心要点提炼	25
13.2	跨章节关联	26
13.3	开放问题	26

1 如何阅读这一讲

CS336: *Language Models From Scratch* 的第一讲同时承担三件事：解释为什么今天仍然值得从零构建语言模型；给出现代 language model 技术栈的历史和课程地图；正式进入第一个技术主题 tokenization。它不是一节只讲行政安排的导论课，而是用课程哲学把后续所有技术单元串起来。

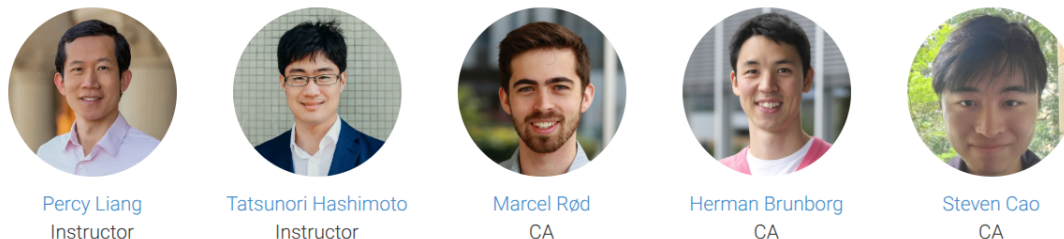


图 1: CS336 Spring 2026 教学团队。

来源: *Stanford CS336 Spring 2026* 官方可执行讲义 `lecture01-slides.py`, 图片 `images/course-staff.png`。

1.1 Spring 2026 的新版定位

这已经是 CS336 第三次开课。新版继续保留“from scratch”的核心哲学，但讲义明确说会更重视 high value-per-time concepts：不要为了实现细节而丢掉整片森林。2026 版还会更多覆盖现代语言模型里的实际成分，例如 mixture of experts、long-context modeling 和 agents。

第一讲的主线

这门课的核心问题不是“怎样调用一个强模型”，而是“给定数据、算力、显存和工程时间，怎样训练并理解一个尽可能好的语言模型”。第一讲从宏观产业背景出发，最后落到 tokenizer 的代码接口，正是为了展示 CS336 的学习路径：先建立全栈视野，再把每个模块拆成可实现、可度量、可优化的工程对象。

1.2 官方材料的形式：可执行讲义

本讲的主要来源不是传统 PDF slides，而是一个 Python 程序 `lecture01-slides.py`。它通过 `text(...)`、`image(...)`、`link(...)` 等函数描述讲义内容，并把少量可运行代码嵌入叙事中。这个形式本身也是课程观点的一部分：现代语言模型课程不应只讲概念，还要让概念和代码处在同一份可检查的材料里。

Listing 1: 官方讲义的入口结构：先建立课程动机，再进入 tokenization

```
1 def main():
2     welcome()
3     why_this_course_exists()
4     current_lm_landscape()
5
6     what_is_this_program()
7
8     course_logistics()
9     course_syllabus()
10
11     tokenization() # First unit
12
13     text("Next time: resource accounting")
```

这个入口顺序很有教学含义：课程不是先抛出 tokenizer 细节，而是先解释为什么研究者需要理解底层技术；不是只列 syllabus，而是把每个单元放在 efficiency 这个总目标下；最后才把第一个具体模块 tokenization 展开。

可执行讲义的价值

可执行讲义让课程材料同时拥有三层结构：第一，顶层函数给出教学大纲；第二，每个函数是一段概念叙事；第三，代码片段可以直接运行、inspect 中间变量。对 CS336 这种强调从零实现的课程来说，这比静态 slides 更接近真实学习方式。

1.3 本章小结

第一讲不是普通导论，而是课程方法论的样板：从产业现实和研究抽象的断裂开始，解释为什么必须理解技术栈，再把课程五个单元和 tokenization 的第一步连接起来。后续学习时，应始终把每个细节放回“固定资源下最大化模型质量”的主问题中理解。

2 为什么今天还要从零理解语言模型

讲义给出的核心问题是：研究者正在越来越远离底层技术。2016 年左右，很多 NLP 研究者会自己实现模型、准备数据、训练并 debug。2018 年以后，BERT 等预训练模型让研究者更多下载模型再 fine-tune。今天，许多研究甚至直接变成 prompt GPT、Claude、Gemini 等 API 模型。

2.1 抽象层上移带来的生产力与风险

抽象层上移当然提高了生产力。调用 API 比自己训练模型快得多，fine-tune 一个预训练模型也比从零训练可靠得多。但语言模型 API 的抽象并不像成熟编程语言或操作系统那样稳定封装。它们会在很多地方“漏出来”：

- 模型上下文长度、tokenizer、system prompt 和 decoding 设置会改变任务行为。
- 训练数据和数据过滤策略会影响知识覆盖、偏见、记忆和评测污染。
- 后训练和安全策略会改变模型愿不愿意回答、怎样回答。

- 推理系统的 KV cache、batching、路由和工具调用会影响延迟、成本和可用能力。
- 闭源模型的版本更新可能让同一个 prompt 的行为发生漂移。

只会使用 API 的局限

如果研究者只看到 API 表面，就很难判断失败来自哪里：是任务定义不清、prompt 不稳、tokenizer 切分异常、训练数据缺失、模型容量不足，还是推理系统约束。CS336 的立场是：要做 fundamental research，就必须能撕开抽象层，理解模型和系统的内部机制。

2.2 从 2016 到今天：研究方式的迁移

阶段	常见工作方式	研究者容易掌握的知识
2016 前后	自己实现并训练模型	数据处理、训练稳定性、模型结构、loss 曲线、硬件瓶颈
2018 前后	下载 BERT 等模型并 fine-tune	下游任务适配、超参数调节、预训练表示的迁移性
今天	prompt 或调用闭源 API	交互设计、prompt 工程、应用封装，但底层机制不透明

表 1: 语言模型研究中抽象层级的变化。

课程并不是否定抽象。好的抽象是技术进步的前提。但当抽象仍在快速变化、底层机制会直接影响科学结论时，研究者不能只停留在最高层。CS336 的哲学是 understanding via building：通过亲手构建 tokenizer、Transformer、optimizer、training loop、systems 和 alignment 组件，获得能跨模型版本迁移的判断力。

从零构建不是怀旧

“From scratch”不是为了重复工业界已经做过的大规模训练，而是为了重新获得因果理解：当 loss 降不下去、GPU 利用率很低、perplexity 异常、模型忽然记不住长上下文时，你知道应该检查哪个模块、用什么指标、怎样设计下一次实验。

2.3 本章小结

今天的语言模型抽象层很高，但仍然高度 leaky。CS336 认为，只会调用强模型不足以支撑基础研究；真正可迁移的能力来自对技术栈内部机制的理解。第一讲因此把“从零构建”作为研究训练，而不是把它当成生产环境复刻。

3 工业化语言模型与课堂可达性的边界

现代 frontier model 已经高度工业化。讲义举了两个尺度感很强的例子：GPT-4 据称训练成本约为 100M USD；xAI 在 2025 年为 Grok 训练建设了 230K GPU 规模的集群。即使这些数字有不确定性，它们仍然说明一个事实：前沿模型的生产方式已经超出普通课程、实验室和个人可复现范围。

3.1 闭源报告与不可复现细节

讲义引用 GPT-4 technical report 的片段，强调前沿模型不会公开足够细节。模型规模、训练数据、训练步数、优化细节、系统并行策略、后训练流程都可能被省略或模糊处理。

2 Scope and Limitations of this Technical Report

This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. **Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.**

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.² We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

图 2: GPT-4 technical report 中关于细节披露限制的片段。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`, 图片 `images/gpt4-no-details.png`。

不能把小模型当作前沿模型的缩小版

课堂可以训练小于 1B 参数的模型，但小模型不一定代表大模型。随着参数量、上下文长度、batch size 和数据规模变化，FLOPs 分布、瓶颈位置、能力曲线和调参直觉都可能改变。因此，CS336 会区分“可迁移的机制”与“可能只在某个尺度成立的经验直觉”。

3.2 尺度改变系统瓶颈

讲义用 attention 与 MLP 的 FLOPs 占比变化提醒学生：即使架构名字仍叫 Transformer，实际计算画像也会随规模变化。某些配置下 attention 更重要，某些配置下 MLP 或通信更重要；长上下文又会进一步改变 prefill 和 decode 的成本结构。

Stephen Roller
@stephenroller

I find people unfamiliar with scaling are shocked by this:

1	description	FLOPs / update	% FLOPs MHA	% FLOPs FFN	% FLOPs attn	% FLOPs logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

8:31 PM · Oct 11, 2022

图 3: 不同模型尺度下 attention 与 MLP FLOPs 占比会发生变化。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`, 图片 `images/roller-flops.png`。

这张图背后的教学信号是：不要把“模型结构”理解成静态图纸。结构一旦落到硬件、数据长度、batching 和服务场景，就会变成资源分配问题。CS336 后续 systems 单元会反复回到这一点。

3.3 尺度改变能力表现

讲义还引用 emergence 相关图，说明某些行为指标可能随规模出现非线性变化。这里更重要的不是争论 emergence 的哲学解释，而是提醒学生：小尺度实验给出的 intuition 有时不能直接外推。

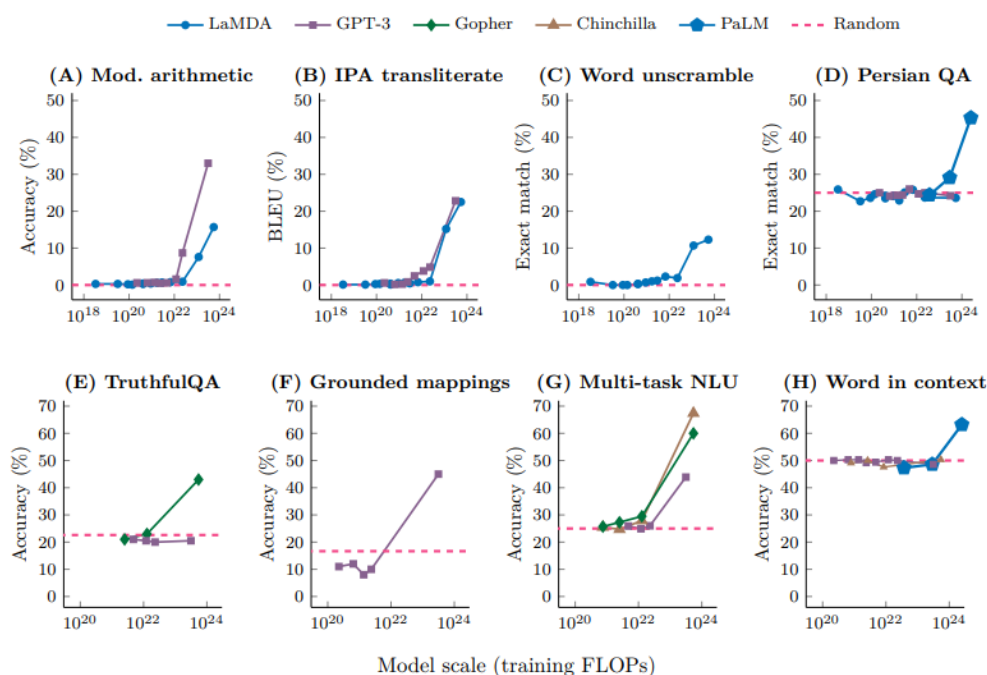


图 4: 能力指标可能随模型尺度出现陡峭变化, 提示小模型直觉需要谨慎外推。

来源: *Stanford CS336 Spring 2026* 官方可执行讲义 `lecture01-slides.py`, 图片 `images/wei-emergence-plot.png`。

3.4 三类可迁移知识

CS336 把课堂能教的知识分成三类: mechanics、mindset 和 intuitions。

类别	含义	迁移性
Mechanics	组件如何工作, 例如 Transformer、BPE、model parallelism、KV cache、optimizer state	高。机制本身可以在小规模实现并检查。
Mindset	如何认真做 resource accounting, 如何压榨硬件, 如何用 scaling 方式设计实验	高。资源意识比具体模型版本更稳定。
Intuitions	哪些数据、架构、训练技巧通常有效, 例如某个激活函数、数据混合比例、后训练 recipe	部分迁移。很多直觉来自实验, 且依赖尺度和系统上下文。

表 2: 第一讲给出的三类知识及其迁移性。

这张表其实是整门课的学习合同: 课程会尽力把 mechanics 和 mindset 讲清楚, 但不会假装所有 scale-specific intuition 都能在课堂规模可靠学习。

4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

图 5: 讲义用 Shazeer 关于 SwiGLU 的表述说明: 一些成功设计首先来自实验, 而不是事前完备理论。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`, 图片 `images/divine-benevolence.png`。

为什么有些 intuition 不容易讲

现代语言模型里有许多选择最初来自经验搜索: 激活函数、normalization placement、数据比例、学习率 schedule、MoE routing 细节等。理论可以事后解释一部分现象, 但在大规模训练中, 很多判断仍来自昂贵实验。课程可以教你怎样设计和解读实验, 却不能凭空提供所有前沿规模的经验。

3.5 Bitter Lesson 的正确读法

讲义把 “The Bitter Lesson” 的常见误读拆开: 错误读法是 “规模就是一切, 算法不重要”; 正确读法是 “能随着资源增长而持续受益的算法最重要”。第一讲把它压缩为一个简单公式:

$$\text{accuracy} = \text{efficiency} \times \text{resources}$$

- accuracy: 模型最终效果, 可以是 loss、perplexity、benchmark 表现或真实任务质量。
- efficiency: 单位资源转化为有效能力的效率, 包括算法效率、系统效率、数据效率和调参效率。
- resources: 可用数据、训练 FLOPs、显存、通信带宽、工程时间和推理预算。

这个公式不是严格物理定律, 而是课程的设计原则。当前沿资源不可得时, 课堂最应该训练的是 efficiency: 同样的 GPU、同样的数据、同样的时间, 谁能得到更好的 loss、更高的吞吐、更稳定的训练。

Bitter Lesson 对 CS336 的意义

大规模并不让算法失去价值, 反而放大了效率的价值。小模型阶段浪费一点算力只是慢几分钟; 前沿模型阶段同样比例的浪费可能意味着数百万美元和数周排队时间。CS336 后续所有单元都可以看成对 efficiency 的不同分解。

3.6 本章小结

前沿模型不可在课堂复现, 但课堂仍然能学习可迁移的 mechanics 和 mindset。尺度会改变系统瓶颈和能力曲线, 因此课程会谨慎处理 intuition。第一讲用 “accuracy = efficiency x resources” 把全课核心目标固定下来: 在资源约束下最大化模型质量。

4 语言模型发展版图

第一讲给出一条从 pre-neural 到 agents 的历史线索。它不是为了背年份，而是为了说明今天的语言模型技术栈如何由许多独立进展叠加而成。

4.1 Pre-neural: 熵、n-gram 与传统系统

早期 language model 主要服务于英语熵估计、机器翻译和语音识别。Shannon 的语言熵工作说明了“预测下一个符号”本身就是理解语言统计结构的基本方式。N-gram 模型则把上下文截断到固定长度，用计数和 smoothing 做概率估计。

从今天看，n-gram 最大的问题是无法学习长距离依赖，也无法共享语义相近片段的统计强度。但它留下了两个核心观念：第一，语言建模可以形式化为序列概率；第二，模型质量可以用 held-out likelihood 或 perplexity 衡量。

4.2 Neural ingredients: 2010 年代的关键积木

2010 年代逐步出现了现代语言模型的关键积木：

- LSTM 让模型能通过门控状态处理较长序列。
- 第一批 neural language model 用分布式表示缓解 n-gram 稀疏性。
- Seq2seq 和 attention 推动机器翻译从 feature engineering 转向端到端学习。
- Adam 优化器降低了大规模神经网络训练的调参难度。
- Transformer 用 self-attention 和高度并行的结构重塑了序列建模。
- Mixture of Experts、GPipe、ZeRO、Megatron-LM 等工作把模型扩展推向系统层。

Transformer 不是孤立发明

Transformer 的成功不是因为一个模块突然替代一切，而是因为它把 attention、并行训练、深层网络优化和硬件友好计算组合到一起。CS336 后续讲架构时，会把这些“积木”拆开，而不是只记住一个 encoder-decoder 图。

4.3 Early foundation models 与 scaling

ELMo、BERT 和 T5 展示了预训练再适配的能力。随后 GPT-2、GPT-3、PaLM、Chinchilla 等模型把 scaling law 推到中心位置。这里的关键转变是：模型开发不再只是“试一个架构”，而是要用小规模实验预测大规模训练结果，用 compute budget 决定参数量、token 数和调参策略。

阶段	代表材料	给 CS336 的启发
早期预训练	ELMo、BERT、T5	预训练表示可以迁移，下游任务不必从零学习语言结构。
拥抱规模	GPT-2、GPT-3、PaLM	规模带来泛化和 in-context learning，但也带来资源规划问题。
Compute-optimal	Chinchilla、scaling laws	参数量和 token 数需要共同优化，不能只盲目增大模型。

表 3: 从 foundation model 到 scaling law 的关键转变。

4.4 开放模型：从 open-weight 到 open development

讲义把开放模型分成几个层次。早期复现 GPT-3 的尝试包括 The Pile、GPT-J、OPT、BLOOM；随后 Llama、Mistral、DeepSeek、Qwen、Kimi、GLM、MiniMax、MIMO 等 open-weight 模型逐渐接近闭源模型；更完整的开放路线则包括 OLMo、Nemotron、Marin 等，它们试图公开更多代码、数据或开发过程。

开放模型为什么对教学重要

CS336 能讲现代语言模型，不只是因为论文公开，更因为开放模型社区让 weights、training recipes、evaluation artifacts、系统实现和失败经验逐渐可见。没有这些公开材料，课程只能停留在概念层，很难带学生做 from-scratch 实现和可验证实验。

4.5 语言模型形态的变化

讲义用四个时间点概括“什么是 language model”的应用形态变化：

年份	典型形态	含义
2018	BERT: something you fine-tune	语言模型是可迁移表示，需要下游任务标注和 fine-tuning。
2020	GPT-3: something you prompt	语言模型开始通过 prompt 在上下文中适配任务。
2022	ChatGPT: something you talk to	对话界面和 instruction following 成为主流交互方式。
2026	Agents: something that acts autonomously	模型开始执行工具调用、长程任务规划和环境交互。

表 4: 语言模型应用形态的演进。

讲义随后强调：形态在变，基础仍然相同。agentic 系统引入了工具、状态、环境和长程任务，但底层仍绕不开 attention、kernel、optimization、data、inference efficiency 和 evaluation。

4.6 本章小结

语言模型的发展不是单线“模型越来越大”，而是统计建模、神经网络结构、优化、数据、系统和开放社区共同推进的结果。今天的 agentic 模型看起来离 tokenizer 很远，但它仍然建立在同一套基础机制上。

5 课程结构：五个单元如何构成语言模型栈

CS336 的 syllabus 围绕五个作业单元组织：basics、systems、scaling laws、data、alignment。第一讲不是简单列目录，而是把这些单元放在 efficiency 的统一目标下。

5.1 Basics：从 tokenizer 到训练循环

第一单元的目标是训练一个基本 language model。它包括三块：tokenization、model architecture 和 training。

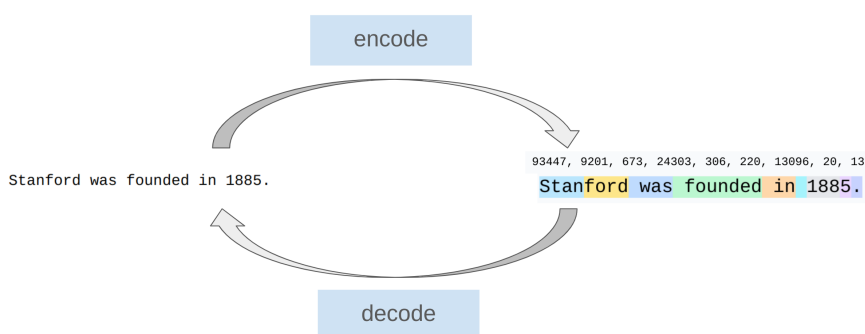


图 6: Tokenization 把原始文本映射到模型可处理的整数序列。

来源：Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`，图片 `images/tokenized-example.png`。

作业 1 要求实现 BPE tokenizer、Transformer、cross-entropy loss、AdamW optimizer 和 training loop，并在 TinyStories 与 OpenWebText 上训练。Leaderboard 的目标不是追求无限资源下的最好结果，而是在固定训练预算下最小化 OpenWebText perplexity。

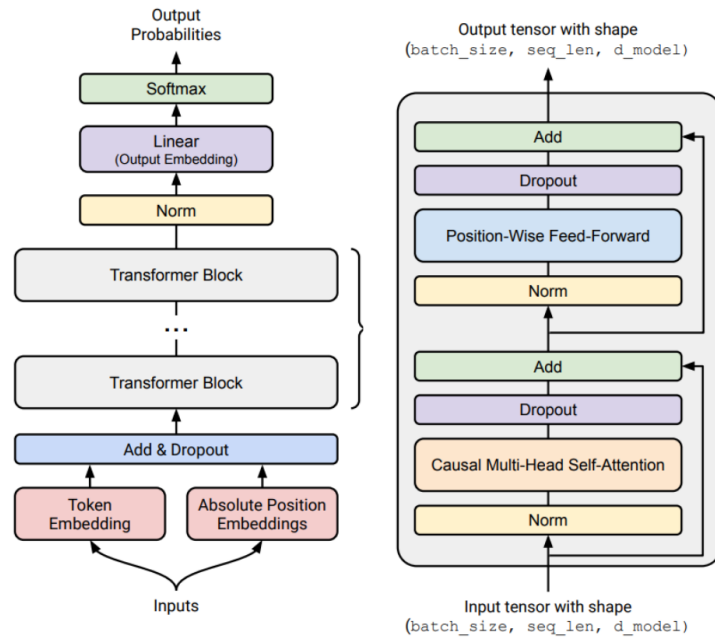


图 7: Transformer 是 basics 单元的起点，但课程会继续讨论现代结构改进。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`, 图片 `images/transformer-architecture.png`.

讲义列出的现代架构改进包括 SwiGLU、RoPE、RMSNorm、QK norm、pre-norm/post-norm、GQA、MLA、sparse/local attention、state-space models、MoE、模型 shape 等。这些不是“技巧清单”，而是三类目标之间的权衡：expressivity、stability、efficiency。

Basics 单元的三角形

一个可用语言模型必须同时满足三点：expressivity，让模型有能力表示复杂依赖；stability，让参数和梯度处在可训练范围；efficiency，让训练和推理能在硬件上跑得足够快。任何单点优化都可能破坏另外两点。

5.2 Systems: 硬件不是背景板

系统单元的目标是 squeeze the most out of the hardware。讲义强调 resource accounting：模型参数必须从 HBM 移到 SM 做计算，kernel launch、memory bandwidth、communication bandwidth 都会限制吞吐。

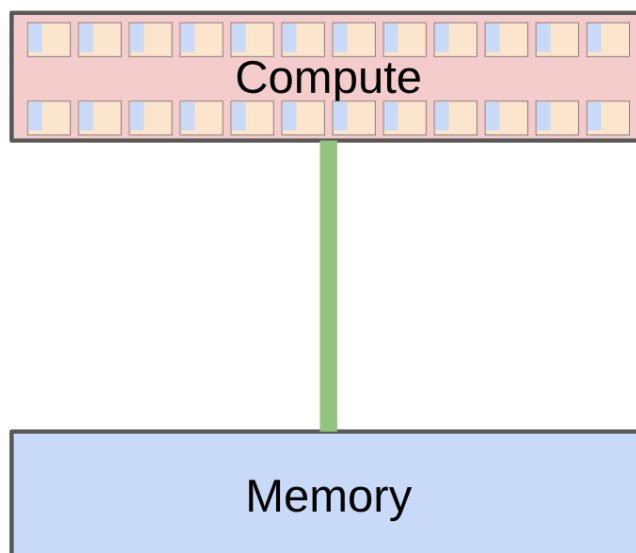


图 8: 计算与内存移动的关系: 参数和激活需要在存储层级与计算单元之间流动。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`, 图片 `images/compute-memory.png`。

CS336 后续会讨论 custom kernels、Triton、roofline analysis、parallelism、distributed training 和 inference。它们共同回答一个问题: 同样的数学运算, 怎样在真实硬件上以更低成本执行?

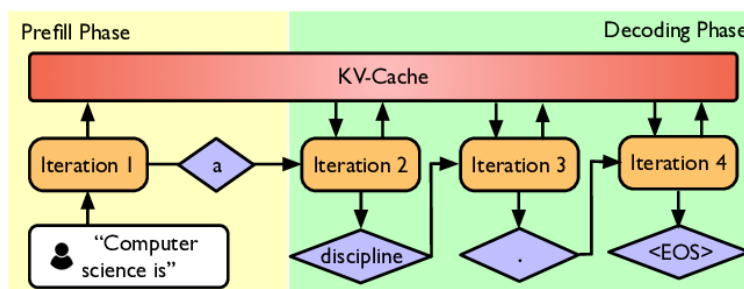


图 9: 推理分为 prefill 与 decode, 两者的并行性和瓶颈不同。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`, 图片 `images/prefill-decode.png`。

Prefill 阶段有完整 prompt, 可以并行处理多个 token, 通常更接近训练时的 compute-bound 计算; decode 阶段一次生成一个 token, 频繁读取 KV cache 和模型权重, 往往更容易 memory-bound。长上下文和 agentic 任务会让这个差异更重要。

5.3 Scaling laws: 用小规模实验规划大规模训练

Scaling 单元让学生学习如何用小实验预测大训练。核心公式之一是训练 FLOPs 的粗略估算:

$$C \approx 6ND$$

- C : 训练计算量, 单位通常为 FLOPs。
- N : 模型参数量。
- D : 训练 token 数。

- 系数 6: 来自 Transformer 训练中前向和反向计算的常用近似, 具体值会随实现和架构细节变化。

这个公式让一个抽象问题变成可规划问题: 给定 C , 应该增大 N , 还是增加 D ? 经典 compute-optimal scaling laws 会通过 isoflop curves 在多个小预算下寻找最优参数量, 再外推到更大预算。

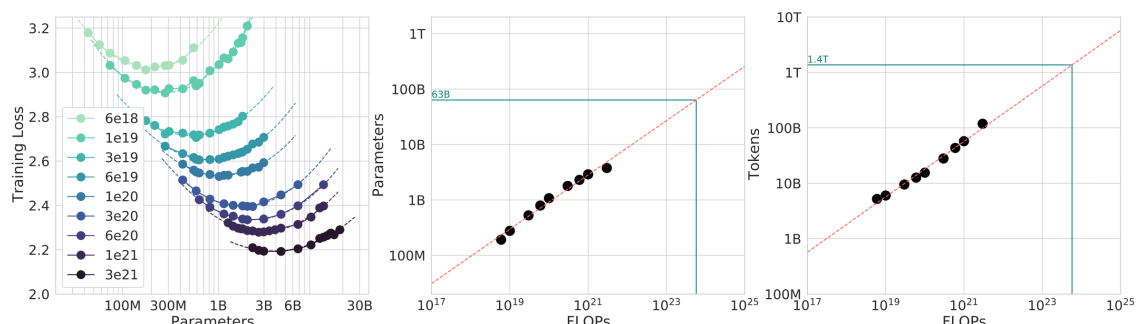


图 10: Chinchilla 风格 isoflop 分析: 在固定 FLOPs 下寻找参数量与 token 数的平衡。

来源: Stanford CS336 Spring 2026 官方可执行讲义 `lecture01-slides.py`, 图片 `images/chinchilla-isoflop.png`。

讲义给出的简化 takeaway 是 $D \approx 20N$, 例如 70B 参数模型大约对应 1.4T tokens。但它也提醒: 这个结论没有直接考虑 inference cost。如果部署成本重要, 可能更偏好较小模型训练更多 token。

5.4 Data: 数据处理是模型质量的一部分

数据单元从一个更根本的问题开始: 我们希望模型具备什么能力? 多语言能力、对话能力、代码能力、agentic coding 能力, 对数据来源和数据处理提出完全不同的要求。

讲义把 data pipeline 拆成 evaluation、curation、transformation、filtering、deduplication、data mixing、rewriting/synthetic data 等环节。每一环都影响 efficiency: 低质量数据会浪费训练 FLOPs; 重复数据会增加 memorization 风险; 错误混合比例会让模型在关键能力上欠训练。

数据不是“更多就好”

在资源约束下, 数据质量和数据匹配度会直接影响训练效率。一个 token 既可以来自高质量代码, 也可以来自乱码网页; 它们消耗相近 FLOPs, 却带来完全不同的学习信号。CS336 把数据处理放在独立单元, 是因为它已经成为语言模型工程的核心部分。

5.5 Alignment: 从 next-token prediction 到弱监督优化

Alignment 单元从一个事实出发: 预训练模型学会了 next-token prediction, 但真实系统还需要遵循指令、拒绝不安全请求、调用工具、完成长程任务。讲义给出的基本模板是:

1. 从模型生成 responses。
2. 用 human、verifier 或 LM judge 给 responses 打分。
3. 更新模型, 让它更偏好高分 responses。

课程会涉及 PPO、DPO 和 GRPO。讲义特别提醒, RL algorithms 在大规模系统中不只是数学公式, 还要求高吞吐 inference、异步 rollouts、on-policy 与系统效率之间的权衡。

5.6 本章小结

五个单元构成一条完整语言模型生产线: basics 给出模型和训练核心; systems 让它跑得快; scaling laws 让大规模实验可预测; data 决定学习信号质量; alignment 把预训练能力转向真实交互。所有单元都服务于同一个问题: 在固定资源下训练和部署最好的模型。

6 Efficiency 作为贯穿全课的设计语言

第一讲反复出现的词是 efficiency。它不是一个口号, 而是贯穿 tokenizer、architecture、training、systems、data、alignment 的共同评价维度。

6.1 资源的多重含义

讲义把 resources 分成 data 和 hardware。hardware 又不只是 FLOPs, 还包括 memory、communication bandwidth、kernel efficiency、集群可用性等。一个训练 recipe 可能在数学上等价, 却因为通信模式不同而在真实集群上慢很多。

资源	典型瓶颈	课程中对应单元
训练 FLOPs	参数量、token 数、前反向计算	scaling laws、architecture、training
显存/HBM	参数、梯度、optimizer state、activations、KV cache	systems、inference
通信带宽	data/tensor/pipeline parallel 的同步与传输	systems、distributed training
数据预算	高质量 token 数、污染、重复、任务覆盖	data、evaluation
人类反馈/验证器	标注成本、judge 偏差、verifier 覆盖	alignment

表 5: 资源不是单一数字, 而是多种约束的组合。

6.2 Tokenization 也是 efficiency 决策

Tokenization 常被当成预处理, 但第一讲把它放进 efficiency lens。原因很直接: Transformer 的 attention 成本随序列长度增长很快。如果 tokenizer 把同样文本切成更多 token, 模型每一步都要处理更长序列, 训练和推理都变贵。

一个粗略的 self-attention 复杂度可以写成:

$$\text{attention cost} = O(n^2 d)$$

- n : token 序列长度。
- d : hidden dimension。
- $O(n^2 d)$: 表示每个 token 需要与其他 token 交互, 序列长度翻倍会显著增加注意力计算。

如果 byte tokenizer 把 1000 bytes 变成约 1000 tokens，而 BPE 把它压成约 250 tokens，那么 attention 相关计算可能相差一个数量级。这个估算不等于完整训练成本，但足以说明 tokenizer 对系统效率的影响。

高压压缩率不一定越高越好

增加 vocabulary size 可以提高 bytes per token，但也会带来稀疏性：每个 token 的训练样本更少，embedding 和输出层更大，罕见 token 更难学。好的 tokenizer 不是简单追求最短序列，而是在覆盖性、压缩率、词表大小和学习难度之间折中。

6.3 本章小结

Efficiency 是 CS336 的统一语言。Tokenization 决定序列长度，systems 决定硬件利用率，scaling laws 决定预算分配，data 决定每个 token 的学习价值，alignment 决定后训练反馈是否高效。第一讲提前建立这套语言，是为了让后续每个模块都能被放回同一个资源约束框架中。

7 Tokenization 的基本定义

Lecture 1 的第一个正式技术单元是 tokenization。原始文本通常表示为 Unicode string，而语言模型在整数 token 序列上建模。因此需要一个 tokenizer 在字符串和 token IDs 之间转换。

7.1 Tokenizer 抽象接口

官方讲义定义了一个非常小的抽象类：

Listing 2: Tokenizer 的抽象接口

```
1 class Tokenizer(ABC):
2     """Abstract interface for a tokenizer."""
3     def encode(self, string: str) -> list[int]:
4         raise NotImplementedError
5
6     def decode(self, indices: list[int]) -> str:
7         raise NotImplementedError
```

这个接口虽小，但包含两个必须满足的要求：

- encode：把原始字符串转成整数序列，让模型可以查 embedding 并进行序列建模。
- decode：把整数序列还原成字符串，让生成结果能回到人类可读文本。

Tokenizer 的本质

Tokenizer 是语言模型的第一层表示学习前处理。它决定模型的“原子单位”：模型不是直接看到字符、byte 或词，而是看到 tokenizer 选择出来的 token IDs。这个选择会影响上下文长度、词表大小、训练稳定性、生成质量和系统成本。

7.2 Round-trip: 可逆性是底线

一个 practical tokenizer 至少应该满足 round-trip:

$$\text{decode}(\text{encode}(x)) = x$$

- x : 原始输入字符串。
- $\text{encode}(x)$: token IDs。
- $\text{decode}(\cdot)$: 把 IDs 映射回字符串的过程。

如果 round-trip 不成立, 模型训练时看到的序列与最终生成文本之间就会出现不可控信息损失。某些特殊 token 或 normalization 规则可能有例外, 但主路径必须可解释、可测试。

7.3 Compression ratio: bytes per token

讲义用 compression ratio 衡量 tokenization 的序列效率:

$$\text{compression ratio} = \frac{\text{number of UTF-8 bytes}}{\text{number of tokens}}$$

- 分子是原始字符串按 UTF-8 编码后的 byte 数。
- 分母是 tokenizer 输出的 token 数。
- 值越大, 表示同样文本被表示成更短 token 序列。

例如字符串 Hello, 加上中文字符会包含 ASCII byte 和多 byte Unicode 字符。character tokenizer 可能 token 数较少但词表巨大, byte tokenizer 词表很小但 compression ratio 固定为 1, BPE 则试图用数据驱动方式合并常见 byte 片段。

7.4 GPT-5 tokenizer 的观察

官方讲义用 OpenAI tiktoken 的 o200k_base 编码器作为例子, 并让学生观察真实 tokenizer 的行为:

- 一个词和它前面的空格常常属于同一个 token, 例如 " world"。
- 同一个词在句首和句中可能对应不同 token, 例如 "hello hello"。
- 数字通常每隔几位切开, 而不是把整个长数字当成一个 token。
- vocabulary size 越大, 潜在 compression ratio 越高, 但 token 统计也越稀疏。

为什么空格会进 token

现代 BPE/byte-level BPE tokenizer 常把空格和后续词片段合在一起, 因为英文中“空格 + 单词”是高频模式。这样做提高压缩率, 也让模型更容易区分词首、词中和句首位置。但它也说明 token 并不等于人类直觉中的 word。

7.5 本章小结

Tokenization 把 Unicode 字符串变成模型可处理的整数序列。一个 tokenizer 至少要可逆，同时要在 compression ratio、vocabulary size 和学习难度之间折中。第一讲接下来用 character、byte、word 三种朴素方案说明为什么需要 BPE。

8 三种朴素 tokenizer 及其失败方式

在进入 BPE 之前，讲义先让学生实现 character tokenizer、byte tokenizer 和 word tokenizer。这个顺序很重要：只有看清朴素方案的失败方式，才能理解 BPE 为什么是一个实用折中。

8.1 Character tokenizer: 直观但词表尴尬

Character tokenizer 把字符串看成 Unicode code points。Python 中可以用 ord 把字符转成整数，用 chr 转回来。

Listing 3: CharacterTokenizer: 用 Unicode code point 表示字符

```
1 class CharacterTokenizer(Tokenizer):
2     """Represent a string as a sequence of Unicode code points."""
3     def encode(self, string: str) -> list[int]:
4         return list(map(ord, string))
5
6     def decode(self, indices: list[int]) -> str:
7         return "".join(map(chr, indices))
```

它的优点是概念直接、round-trip 容易测试。问题是 Unicode 字符空间很大，约有 150K 级别字符；同时很多字符极其罕见，模型需要为稀有字符保留 embedding 和输出概率，却很少得到训练样本。

8.2 Byte tokenizer: 覆盖性强但序列太长

Byte tokenizer 把字符串先编码成 UTF-8 bytes，每个 byte 是 0 到 255 的整数。

Listing 4: ByteTokenizer: 先转 UTF-8 bytes，再把 byte 当 token

```
1 class ByteTokenizer(Tokenizer):
2     """Represent a string as a sequence of bytes."""
3     def encode(self, string: str) -> list[int]:
4         string_bytes = string.encode("utf-8")
5         indices = list(map(int, string_bytes))
6         return indices
7
8     def decode(self, indices: list[int]) -> str:
9         string_bytes = bytes(indices)
10        string = string_bytes.decode("utf-8")
11        return string
```

Byte tokenizer 的词表非常干净：只需要 256 个基础值，且能覆盖所有 Unicode 文本。问题是 compression ratio 固定为 1 byte/token。对 Transformer 来说，这会让序列过长，attention 和 KV cache 成本都变高。

方案	优势	主要问题
Character tokenizer	与人类字符直觉接近, 容易可逆	Unicode 空间大, 稀有字符多, 词表利用率低
Byte tokenizer	词表固定为 256, 覆盖所有 UTF-8 文本	序列太长, compression ratio 低, attention 成本高
Word tokenizer	token 语义强, 压缩率通常较好	词表无界, 稀有词和未登录词处理困难

表 6: 三种朴素 tokenizer 的核心取舍。

8.3 Word tokenizer: 语义自然但开放词表难处理

Word tokenizer 更接近传统 NLP: 用规则把字符串切成词和标点。讲义示例使用正则规则, 让连续字母数字组成词, 其余字符单独保留。

Listing 5: 朴素 word tokenizer 的切分思路

```

1 string = "I'll say supercalifragilisticexpialidocious!"
2 chunks = regex.findall(r"\w+|.", string)
3 # 可能得到: ["I", "'", "ll", " ", "say", " ", "supercalifragilisticexpialidocious", "!"]

```

它的问题来自 open vocabulary。训练集中没见过的新词怎么办? 如果统一映射为 UNK, 就丢失了拼写信息, 也会污染 perplexity 计算。如果把所有词都加入词表, 词表会巨大且长尾严重。

UNK token 的代价

UNK 看似解决了未登录词, 实际上把所有未知词压成同一个符号。模型无法区分一个新的人名、一个新代码标识符、一个拼写错误和一个专业术语。对开放域语言模型来说, 这种信息损失很难接受。

8.4 一个具体例子: 三种方案的数值直觉

考虑字符串 Hello,、一个地球 emoji、中文 你好和感叹号的混合文本。按 UTF-8 编码, ASCII 字符通常占 1 byte, 中文字符通常占 3 bytes, emoji 可能占 4 bytes。于是:

- character tokenizer 的 token 数接近人类看到的字符数, 但词表必须覆盖很大的 Unicode 空间。
- byte tokenizer 的 token 数等于 UTF-8 bytes 数, 覆盖性完美但序列最长。
- word tokenizer 对英文词可能压缩良好, 但对中文、emoji、代码和未见词需要额外规则。

这个例子说明, tokenizer 不是纯语言学问题, 而是表示效率问题。语言模型需要一个既不丢信息、又不让序列爆炸、还不让词表爆炸的中间方案。

8.5 本章小结

Character tokenizer、byte tokenizer 和 word tokenizer 分别占据三个极端: 字符直观但词表大, byte 覆盖性强但序列长, word 语义自然但开放词表困难。BPE 的动机正是把 byte 的覆盖性和 chunk 的压缩效率结合起来。

9 Byte Pair Encoding: 从压缩算法到语言模型 tokenizer

BPE 最初由 Philip Gage 在 1994 年作为数据压缩算法提出，后来被 Sennrich 等人引入 neural machine translation，又被 GPT-2 等模型采用。它的核心思想很简单：从 byte 级 token 开始，反复合并最常见的相邻 token pair。

9.1 BPE 的基本数据结构

官方讲义把 BPE 参数写成一个 dataclass:

Listing 6: BPE tokenizer 需要的两张表

```
1 @dataclass(frozen=True)
2 class BPETokenizerParams:
3     """All you need to specify a BPETokenizer."""
4     vocab: dict[int, bytes]          # index -> bytes
5     merges: dict[tuple[int, int], int] # index1, index2 -> new_index
```

- vocab 告诉 decode: 每个 token ID 对应哪段 bytes。
- merges 告诉 encode: 哪些相邻 token pair 应该被合并成新 token。

初始时, vocab 包含 256 个 byte token。每做一次 merge, 就新增一个 token ID, 例如 256、257、258, 并把它对应到两个旧 token 的 byte 拼接。

BPE 的关键折中

BPE 保留 byte-level fallback, 所以不会遇到真正无法表示的字符串; 同时它把高频 byte 序列合成更长 chunk, 提高 compression ratio。它不是最优分词算法, 但在覆盖性、效率和实现复杂度之间取得了非常实用的平衡。

9.2 merge 函数的细节

讲义给出一个简单但重要的 merge 函数:

Listing 7: 把所有目标 pair 替换为新 token

```
1 def merge(indices: list[int], pair: tuple[int, int], new_index: int) -> list[int]:
2     """Return `indices`, but with all instances of `pair` replaced with `new_index`."""
3     new_indices = []
4     i = 0
5     while i < len(indices):
6         if i + 1 < len(indices) and indices[i] == pair[0] and indices[i + 1] == pair[1]:
7             new_indices.append(new_index)
8             i += 2
9         else:
10            new_indices.append(indices[i])
11            i += 1
12    return new_indices
```


这个循环里最容易忽略的是 `i += 2`。一旦匹配到 pair，两个旧 token 被一个新 token 替代，扫描位置必须跳过两个元素，避免重叠合并。例如序列 `[a, a, a]` 合并 pair `(a, a)` 时，结果应是 `[aa, a]`，而不是不受控地同时合并第 1-2 和第 2-3 个 token。

BPE merge 的 off-by-one 风险

实现 BPE 时，很多 bug 不来自算法概念，而来自扫描边界：最后一个 token 没有右邻居；匹配后要跳过两个位置；不匹配时只前进一步；pair 的合并顺序必须保持训练时的顺序。Assignment 1 的单元测试通常会抓这些细节。

9.3 训练 BPE：统计相邻 pair

训练过程如下：

1. 把训练字符串编码成 UTF-8 bytes。
2. 统计当前 token 序列中所有相邻 pair 的出现次数。
3. 找出出现次数最多的 pair。
4. 给它分配一个新 token ID，并更新 vocab 和 merges。
5. 调用 merge 替换序列中的所有该 pair。
6. 重复指定次数。

Listing 8: 教学版 BPE 训练循环

```
1 def train_bpe(string: str, num_merges: int) -> BPETokenizerParams:
2     indices = list(map(int, string.encode("utf-8")))
3     merges: dict[tuple[int, int], int] = {}
4     vocab: dict[int, bytes] = {x: bytes([x]) for x in range(256)}
5
6     for i in range(num_merges):
7         counts = count_adjacent_pairs(indices)
8         pair = max(counts, key=counts.get)
9
10        new_index = 256 + i
11        merges[pair] = new_index
12        vocab[new_index] = vocab[pair[0]] + vocab[pair[1]]
13        indices = merge(indices, pair, new_index)
14
15    return BPETokenizerParams(vocab=vocab, merges=merges)
```

9.4 Worked example: the cat in the hat

讲义用 the cat in the hat 训练 3 次 merge。初始 byte 序列对应字符：

t h e _ c a t _ i n _ t h e _ h a t

每次选择最高频相邻 pair，并把它变成新 token：

轮次	被合并 pair	新 token	合并后的 token 片段
1	t + h	256 = th	th e _c a t _i n _th e _h a t
2	th + e	257 = the	the _c a t _i n _the _h a t
3	the + space	258 = the_	the_c a t _i n _the_h a t

表 7: BPE 在短字符串上的三次 merge 示例。

这个例子有两点值得注意。第一，BPE 不知道英文语法，只看相邻 token pair 的频率；the 和后面的空格合并，是因为 the_ 在语料中高频。第二，merge 后的 token 不一定等于完整单词，也可能是前缀、后缀、空格加词、词的一部分或跨脚本片段。

9.5 使用 BPE tokenizer

训练得到 params 后，编码新字符串时先转成 bytes，再按 merges 的顺序应用合并：

Listing 9: 教学版 BPE encode/decode

```

1 class BPETokenizer(Tokenizer):
2     """BPE tokenizer given a set of merges and a vocabulary."""
3     def __init__(self, params: BPETokenizerParams):
4         self.params = params
5
6     def encode(self, string: str) -> list[int]:
7         indices = list(map(int, string.encode("utf-8")))
8         # Note: this is a very slow implementation
9         for pair, new_index in self.params.merges.items():
10             indices = merge(indices, pair, new_index)
11         return indices
12
13     def decode(self, indices: list[int]) -> str:
14         bytes_list = list(map(self.params.vocab.get, indices))
15         string = b"".join(bytes_list).decode("utf-8")
16         return string

```

这里的 encode 很慢，因为它对每个输入都遍历所有 merges。真实 tokenizer 会用更高效的数据结构，只处理当前序列中可能发生的合并。

BPE 是“训练出来的 tokenizer”

BPE 的 vocabulary 和 merges 来自训练语料，而不是人工语言学词典。因此，不同语料、不同特殊 token 策略、不同词表大小会产生不同 tokenization。模型的 tokenizer 一旦固定，就成为模型接口的一部分，不能随意替换。

9.6 本章小结

BPE 从 byte 级表示出发，反复合并高频相邻 pair，得到一个既可覆盖任意文本、又能压缩常见片段的 tokenizer。理解 BPE 的关键不是背算法名字，而是理解 vocab、merges、merge 顺序和 round-trip 如何共同保证编码与解码。

10 Assignment 1 中 tokenizer 实现的工程细节

官方讲义明确提醒：课堂示例是慢速教学版。Assignment 1 要求学生实现更接近真实系统的 BPE tokenizer，并处理 special tokens、pre-tokenization 和速度优化。

10.1 不要每次遍历所有 merges

教学版 `encode` 对每个输入遍历所有 merges。如果 vocabulary 有几十万 token，merges 也可能有几十万条，这样的实现会非常慢。更好的思路是维护当前序列中的相邻 pair，只更新受一次 merge 影响的局部邻居。

实现策略	优点	问题
遍历所有 merges	概念简单，容易验证正确性	对长文本和大词表极慢
维护 pair ranks	只选择当前存在且 rank 最小的 pair	数据结构复杂，需要处理局部更新
预分词后逐段 BPE	减少跨类别合并，符合 GPT 系 tokenizer 实践	regex 和 special token 规则需要非常仔细

表 8: BPE encode 实现中的常见策略取舍。

10.2 Special tokens 不能被普通规则吞掉

语言模型通常需要特殊 token，例如 `<|endoftext|>`、padding token、chat template 中的角色标记或工具调用标记。这些 token 不应该被普通 BPE 规则切碎，否则模型接口语义会被破坏。

特殊 token 是协议，不只是字符串

`<|endoftext|>` 看起来像普通字符序列，但在模型协议里它有边界语义。如果 tokenizer 把它切成多个普通 token，训练和推理就可能无法正确表示文档结束、对话边界或工具调用结构。

10.3 Pre-tokenization: 先切粗块，再做 BPE

GPT-2 风格 tokenizer 会先用 regex 做 pre-tokenization，把文本拆成较粗的片段，再在每个片段内部做 byte-level BPE。这样可以减少一些不自然的跨类别合并，也让空格、数字、标点、单词边界的处理更可控。

Pre-tokenization 的重点不是追求“语言学上最正确的分词”，而是给 BPE 一个稳定的统计边界。对多语言、代码和数学文本来说，regex 规则会直接影响模型看到的 token 结构。

10.4 Decode 的错误处理

BPE 的 `decode` 通常先把 token IDs 映射回 bytes，再用 UTF-8 解码。这里可能遇到非法 byte 序列、未知 token ID 或特殊 token。教学版代码默认输入合法，真实系统必须明确错误策略：抛异常、替换为 replacement character，还是保留特殊 token 字面量。

Tokenizer 是模型 API 的一部分

训练模型时的 tokenizer、推理服务的 tokenizer、评测脚本的 tokenizer 必须一致。换 tokenizer 等于换输入空间；即使模型权重不变，embedding 对应关系也会被破坏。因此，tokenizer 文件通常和 model checkpoint 一起发布。

10.5 本章小结

Assignment 1 的 tokenizer 不只是算法练习。它要求学生面对真实系统问题：速度、special token 协议、pre-tokenization、错误处理和可复现性。一个慢速但正确的 BPE 能帮助理解概念，但一个可用的 tokenizer 必须把这些工程细节处理好。

11 Tokenizer-free 梦想与现实约束

讲义在 tokenization 部分也提到一个更长远的方向：tokenizer-free architectures。理想上，模型可以直接处理 bytes，不再需要人为设计 tokenizer。相关路线包括 ByT5、MEGABYTE、BLT、T-Free、H-Net 等。

11.1 为什么直接建模 bytes 很诱人

Byte-level 模型有几个理论优势：

- 不需要处理未登录词，任意文本都可表示。
- 不把 tokenizer 作为固定前处理，减少手工归纳偏置。
- 对多语言、代码、emoji、噪声文本和新符号更统一。
- 可以避免 tokenizer 与模型能力之间的接口错配。

但它的成本也非常明显：byte 序列长，attention 和 KV cache 成本高；模型需要从更底层符号学习 chunk；在当前主流 Transformer 架构和硬件约束下，还没有在 frontier scale 成为主流。

11.2 BPE 为什么仍然实用

BPE 不是最终答案，却是今天很强的工程折中。它把常见片段压成较短序列，让 Transformer 在有限上下文和有限 FLOPs 下学习更高层模式；同时又保留 byte fallback，避免 word tokenizer 的 open vocabulary 问题。

tokenizer-free 与 BPE 的关系

tokenizer-free 是一个重要研究方向，但当前课程从 BPE 开始是务实选择：它是许多现代语言模型实际采用的机制，也是理解资源效率、上下文长度和模型接口的最好入口之一。学懂 BPE，并不意味着相信 BPE 永远最优；它意味着理解当前系统为什么这样设计。

11.3 本章小结

直接从 bytes 学习是优雅目标，但今天仍受序列长度和计算效率限制。BPE 的价值在于把覆盖性和压缩率结合起来，让主流 Transformer 在当前硬件约束下更可行。未来 tokenizer-free 模型如果成为主流，也仍然要回答同一个问题：如何在原始信号上形成高效的可学习 chunk。

12 拓展阅读

- CS336 Spring 2026 课程主页: <https://cs336.stanford.edu/>。
- Stanford CS336 课程材料仓库: <https://github.com/stanford-cs336/lectures>。
- Sennrich et al., 2016, *Neural Machine Translation of Rare Words with Subword Units*: BPE 在神经机器翻译中的经典应用。
- Kaplan et al., 2020 与 Chinchilla, 2022: scaling laws 与 compute-optimal training 的基础阅读。
- Karpathy 的 tokenization 视频: <https://www.youtube.com/watch?v=zduSFxRajkE>。

12.1 本章小结

第一讲引用的材料横跨 tokenizer、scaling laws、开放模型和现代架构。拓展阅读的重点不是一次读完所有论文，而是在后续学习中把每个技术单元和原始论文、官方作业、开放实现对应起来。

13 总结与延伸

Lecture 1 的真正作用是建立一套学习语言模型的坐标系。它先指出研究者与底层技术脱节的问题，再承认前沿模型工业化带来的不可复现边界；随后用 mechanics、mindset、intuitions 区分哪些知识能在课堂规模迁移，哪些经验必须谨慎外推；最后把全课结构落到第一个具体模块 tokenization。

13.1 核心要点提炼

1. CS336 的 from-scratch 哲学是为了获得因果理解，而不是为了复刻前沿模型。
2. 前沿模型昂贵且闭源，因此课堂重点是可迁移 mechanics 和 resource mindset。
3. Bitter Lesson 的正确读法是：可扩展算法和效率最重要，而不是算法不重要。
4. 语言模型技术栈由历史积木累积而成：n-gram、LSTM、attention、Transformer、scaling laws、开放模型、agents。
5. 五个课程单元围绕 efficiency 展开：basics、systems、scaling laws、data、alignment。
6. Tokenizer 是模型的第一层接口，直接影响序列长度、词表大小、训练效率和生成可逆性。
7. BPE 是一个实用 heuristic：从 bytes 出发，合并高频相邻 pair，在覆盖性和压缩率之间折中。

13.2 跨章节关联

Tokenization 和 systems 看似相距很远，其实通过序列长度连接：token 数越多，attention、KV cache、decode latency 越贵。Tokenization 和 data 也相连：BPE merges 来自训练语料，语料分布会改变 token 统计。Tokenization 和 alignment 同样相关：chat template、special tokens、tool-call markers 都需要 tokenizer 协议稳定支持。

因此，第一讲的一个隐藏主题是：语言模型没有“纯局部模块”。一个 tokenizer 决策会影响 architecture；一个 architecture 决策会影响 systems；一个 systems 瓶颈会反过来限制 scaling recipe；一个数据决策会改变模型能力；一个 alignment 决策会改变推理负载。

13.3 开放问题

- Tokenizer-free 模型能否在 frontier scale 上同时保持 byte-level 覆盖性和高效率？
- 如果 agentic 系统大量使用工具调用和结构化 trace，tokenizer 应该如何表示代码、JSON、终端输出和长上下文状态？
- Scaling laws 在后训练、agentic trajectories 和 inference-time compute 中是否仍然有同样强的预测性？
- 开放模型社区能否公开足够多的训练细节，让课程级复现与前沿经验之间的距离继续缩小？

最终 takeaway

第一讲把 CS336 的学习方式压缩成一句话：在固定资源下，用可实现、可测量、可解释的方式训练更好的语言模型。Tokenization 是这条路的第一步，因为它决定模型从什么“原子”开始看世界；后续课程会继续沿着 efficiency 这条线，把架构、训练、系统、数据和对齐逐一拆开。